BCSE301L SOFTWARE ENGINEERING

Module:5 Validation And Verification

Strategic Approach to Software Testing, Testing Fundamentals Test Plan, Test Design, Test Execution, Reviews, Inspection and Auditing – Regression Testing – Mutation Testing - Object oriented testing - Testing Web based System - Mobile App testing – Mobile test Automation and tools – DevOps Testing – Cloud and Big Data Testing

Test plan(procedures)

- A test plan is a detailed document which describes software testing areas and activities. It outlines the test strategy, objectives, test schedule, required resources (human resources, software, and hardware), test estimation and test deliverables.
- The test plan is a template for conducting software testing activities as a defined process that is fully monitored and controlled by the testing manager. The test plan is prepared by the Test Lead (60%), Test Manager(20%), and by the test engineer(20%).

Types of Test Plan

There are three types of the test plan

- Master Test Plan
- Phase Test Plan
- Testing Type Specific Test Plans

Master Test Plan

- Master Test Plan is a type of test plan that has multiple levels of testing. It includes a complete test strategy.

Phase Test Plan

- A phase test plan is a type of test plan that addresses any one phase of the testing strategy. For example, a list of tools, a list of test cases, etc.

Specific Test Plans

- Specific test plan designed for major types of testing like security testing, load testing, performance testing, etc. In other words, a specific test plan designed for non-functional testing.

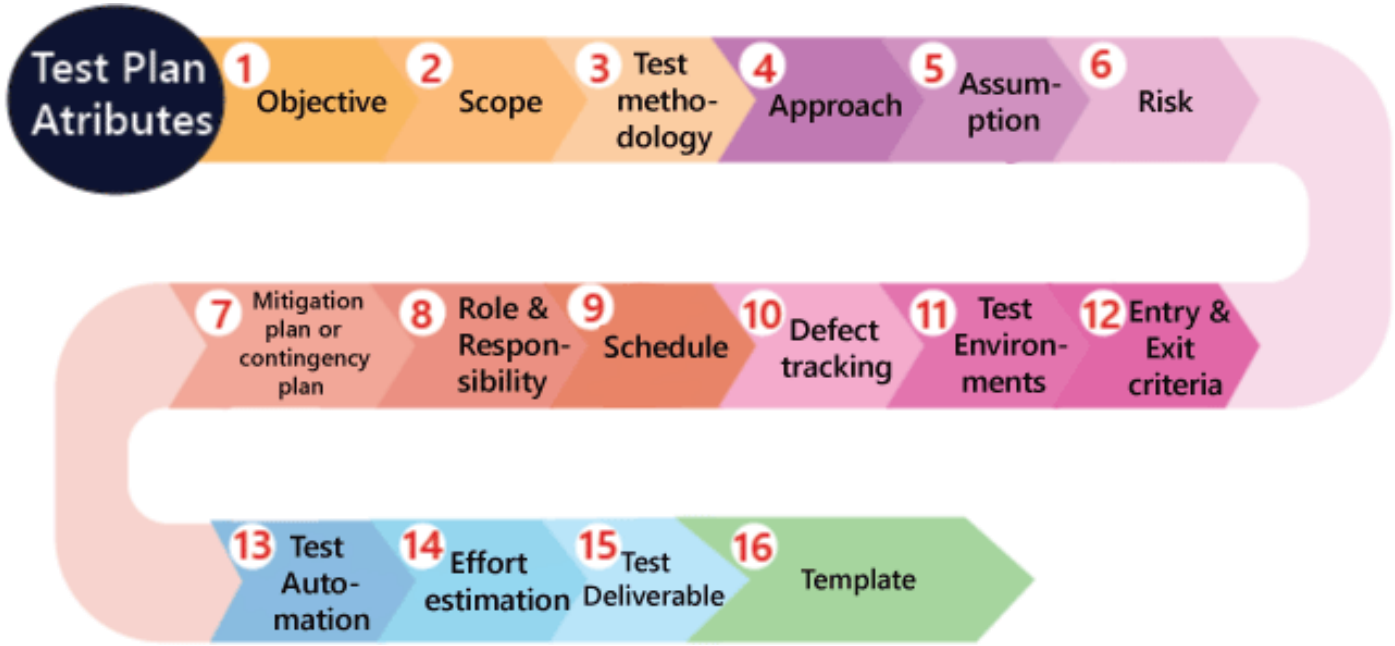
How to write a Test Plan

Making a test plan is the most crucial task of the test management process. seven steps to prepare a test plan.

- First, analyze product structure and architecture.
- Now design the test strategy.
- Define all the test objectives.
- Define the testing area.
- Define all the useable resources.
- Schedule all activities in an appropriate manner.
- Determine all the Test Deliverables.

Test plan components or attributes

The test plan consists of various parts, which help us to derive the entire testing activity.



Test Plan Template

Below find important constituents of a test plan-

- 1 Introduction
- 1.1 Scope
  - 1.1.1 In Scope
  - 1.1.2 Out of Scope
- 1.2 Quality Objective
- 1.3 Roles and Responsibilities
- 2 Test Methodology
- 2.1 Overview
- 2.2 Test Levels
- 2.3 Bug Triage
- 2.4 Suspension Criteria and Resumption Requirements
- 2.5 Test Completeness
- 3 Test Deliverables
- 4 Resource & Environment Needs
- 4.1 Testing Tools
- 4.2 Test Environment

- **Objectives:** It consists of information about modules, features, test data etc., which indicate the aim of the application means the application behavior, goal, etc.
- **Scope:** It contains information that needs to be tested with respective of an application. The Scope can be further divided into two parts:

In scope

Out scope

- **In scope:** These are the modules that need to be tested rigorously (in-detail).
- **Out scope:** These are the modules, which need not be tested rigorously.
- **For example**, Suppose we have a Gmail application to test, where **features to be tested** such as **Compose mail, Sent Items, Inbox, Drafts** and the **features which not be tested** such as **Help**, and so on which means that in the planning stage, we will decide that which functionality has to be checked or not based on the time limit given in the product.

Quality Objective

- Here make a mention of the overall objective that you plan to achieve with your manual testing and automation testing.

Some objectives of your testing project could be

- Ensure the Application Under Test (AUT)conforms to functional and non-functional requirements
- Ensure the AUT meets the quality specifications defined by the client
- Bugs/issues are identified and fixed before go live

Roles and Responsibilities

Detail description of the Roles and responsibilities of different team members like

- QA Analyst
- Test Manager
- Configuration Manager
- Developers
- Installation Team

2.2) Test Levels

Test Levels define the Types of Testing to be executed on the Application Under Test (AUT). The Testing Levels primarily depends on the scope of the project, time and budget constraints.

2.3) Bug Triage

The goal of the triage is to

- To define the type of resolution for each bug
- To prioritize bugs and determine a schedule for all “To Be Fixed Bugs’.

2.4) Suspension Criteria and Resumption Requirements

- Suspension criteria define the criteria to be used to suspend all or part of the testing procedure while Resumption criteria determine when testing can resume after it has been suspended.

2.5) Test Completeness

Here you define the criteria's that will deem your testing complete.

- For instance, a few criteria to check Test Completeness would be 100% test coverage
- All Manual & Automated Test cases executed
- All open bugs are fixed or will be fixed in next release

3) Test Deliverables

- Here mention all the Test Artifacts that will be delivered during different phases of the testing lifecycle.

Here are the simple deliverables

- Test Plan
- Test Cases
- Requirement Traceability Matrix
- Bug Reports
- Test Strategy
- Test Metrics
- Customer Sign Off

Requirement no	Module name	High level requirement	Low level requirement	Test case name

	A	B	C	D	E
1	RTM Template				
2	**Requirement number**	**Module number**	**High level requirement**	**Low level requirement**	**Test case name**
3	2	Loan	**2.1 Personal loan**	2.1.1--> personal loan for private employee	beta-2.0-personal loan
4				2.1.2--> personal loan for government employee	
5				2.1.3--> personal loan for jobless people	
6			**2.2 Car loan**	2.2.1--> car loan for private employee	
7				——	
8			**2.3 Home loan**	——	
9				——	
10				——	
11					

2) Test Methodology

2.1) Overview

Mention the reason of adopting a particular test methodology for the project. The test methodology selected for the project could be

- Waterfall
- Iterative
- Agile
- Extreme Programming

4) Resource & Environment Needs

4.1) Testing Tools

Make a list of Tools like

- Requirements Tracking Tool
- Bug Tracking Tool
- Automation Tools

4.2) Test Environment

- It mentions the minimum **hardware** requirements that will be used to test the Application.

Following **software's** are required in addition to client-specific software.

- Windows 8 and above
- Office 2013 and above
- MS Exchange, etc.

Sample Test Plan Document Banking Web Application Example

1 Introduction

- The Test Plan is designed to prescribe the scope, approach, resources, and schedule of all testing activities of the project.
- The plan identify the items to be tested, the features to be tested, the types of testing to be performed, the personnel responsible for testing, the resources and schedule required to complete testing, and the risks associated with the plan.
- **1.1 Scope**
- **1.1.1 In Scope**
- All the feature of the Banking web application which were defined in software requirement and the specifications are to be tested

Module Name	Applicable Roles	Description
Balance Enquiry	Manager Customer	**Customer**: A customer can have multiple bank accounts. He can view balance of his accounts only **Manager**: A manager can view balance of all the customers who come under his supervision
Fund Transfer	Manager Customer	**Customer:** A customer can have transfer funds from his “own” account to any destination account. **Manager**: A manager can transfer funds from any source bank account to destination account
Mini Statement	Manager Customer	A Mini statement will show last 5 transactions of an account **Customer:** A customer can see mini-statement of only his “own” accounts **Manager:** A manager can see mini-statement of any account

Customized Statement	Manager Customer	A customized statement allows you to filter and display transactions in an account based on date, transaction value **Customer:** A customer can see Customized- statement of only his “own” accounts **Manager**: A manager can see Customized -statement of any account
Change Password	Manager Customer	**Customer:** A customer can change password of only his account. **Manager**: A manager can change password of only his account. He cannot change passwords of his customers
New Customer	Manager	**Manager**: A manager can add a new customer.
	Manager	**Manager:** A manager can edit details like address, email, telephone of a customer.

New Account	Manager	A customer can have multiple saving accounts (one in his name, other in a joint name etc). He can have multiple current accounts for different companies he owns. Or he can have a multiple current and saving accounts. **Manager:** A manager can add a new account for an existing customer.
Edit Account	Manager	**Manager:** A manager can add a edit account details for an existing account
Delete Account	Manager	**Manager:** A manager can add a delete an account for a customer.
Delete Customer	Manager	A customer can be deleted only if he/she has no active current or saving accounts **Manager:** A manager can delete a customer.
Deposit	Manager	**Manager:** A manager can deposit money into any account. Usually done when cash is deposited at a bank branch.
Withdrawal	Manager	**Manager:** A manager can withdraw money from any account. Usually done when cash is withdrawn at a bank branch.

1.1.2 Out of Scope

These feature are not be tested because they are not included in the software requirement specs

- User Interfaces
- Hardware Interfaces
- Software Interfaces
- Database logical
- Communications Interfaces
- Website Security and Performance

1.2 Quality Objective

- The test objectives are to **verify** the Functionality of bank website, the project should focus on testing the **banking operation** such as Account Management, Withdrawal, and Balance.

• 1.3 Roles and Responsibilities

No.	Member	Tasks
1.	Test Manager	Manage the whole project Define project directions Acquire appropriate resources
2.	Test	Identifying and describing appropriate test techniques/tools/automation architecture Verify and assess the Test Approach Execute the tests, Log results, Report the defects. Outsourced members
3.	Developer in Test	Implement the test cases, test program, test suite etc.
4.	Test Administrator	Builds up and ensures test environment and assets are managed and maintained Support Tester to use the test environment for test execution
5.	SQA members	Take in charge of quality assurance Check to confirm whether the testing process is meeting specified requirements

2 Test Methodology

2.1 Overview

2.2 Test Levels

- In the project, there’re 3 types of testing should be conducted.
- **Integration** Testing (Individual software modules are combined and tested as a group)
- **System** Testing: Conducted on a **complete**, **integrated** system to evaluate the system’s compliance with its specified requirements
- **API testing:** Test all the APIs create for the software under tested

2.3 Bug Triage

2.4 Suspension Criteria and Resumption Requirements

- If the team members report that there are **40%** of test cases **failed**, suspend testing until the development team fixes all the failed cases.

2.5 Test Completeness

- Specifies the criteria that denote a **successful** completion of a test phase
- **Run** rate is mandatory to be **100%** unless a clear reason is given.
- **Pass** rate is **80%,** achieving the pass rate is **mandatory**

2.6 Project task and estimation and schedule

Task	Members	Estimate effort
Create the test specification	Test Designer	170 man-hour
Perform Test Execution	Tester, Test Administrator	80 man-hour
Test Report	Tester	10 man-hour
Test Delivery		20 man-hour
Total		280 man-hour

Schedule to complete these tasks

3 .Test Deliverables

Test deliverables are provided as below

Before testing phase

- Test plans document.
- Test cases documents
- Test Design specifications.

During the testing

- – Test Tool Simulators.
- – Test Data
- – Test Trace-ability Matrix – Error logs and execution logs.

After the testing cycles is over

- Test Results/reports
- Defect Report
- Installation/ Test procedures guidelines
- Release notes

4 Resource & Environment Needs

4.1 Testing Tools

No.	Resources	Descriptions
1.	Server	Need a Database server which install MySQL server Web server which install Apache Server
2.	Test tool	Develop a Test tool which can auto generate the test result to the predefined form and automated test execution
3.	Network	Setup a LAN Gigabit and 1 internet line with the speed at least 5 Mb/s
4.	Computer	At least 4 computer run Windows 7, Ram 2GB, CPU 3.4GHZ

4.2 Test Environment

- It mentions the minimum hardware and software requirements that will be used to test the Application.
- Following software’s are required in addition to client-specific software.
- Windows 11 and above
- Office 2021 and above
- MS Exchange, etc.

Test Design

- Test design is a process that describes “how” testing should be done. It includes processes for the identifying test cases by enumerating steps of the defined test conditions.
- The test cases may be linked to the test conditions and project objectives directly or indirectly depending upon the methods used for test monitoring, control and traceability.
- The objectives consist of test objectives, strategic objectives and stakeholder definition of success.

When to create test design?

- After the test conditions are defined and sufficient information is available to create the test cases of high or low level, test design for a specified level can be created.
- For lower level testing, test analysis(what is to be tested) and design(test cases) are combined activity. For higher level testing, test analysis is performed first, followed by test design.
- There are some activities that routinely take place when the test is implemented. These activities may also be incorporated into the design process when the tests are created in an iterative manner.

Test Implementation

- Test implementation is the practice of organizing and prioritizing tests. This is carried out by Test Analysts who implement the test designs as real test cases, test processes and test data.
- If you are observing the IEEE 829 standard, you must define these parameters as well :
- Test inputs
- Expected results for each **test case**
- Steps to be followed for each **test process**

Some of the checks that could be performed to confirm that the team ready to execute tests include:

- Ensuring that the test environment is in place
- Ensuring every test case is well documented and reviewed
- Putting test environment in a state of readiness
- Checking against explicit and implicit entry criteria for the specified test level
- Describing test environment as well as test data in great detail
- Performing code acceptance check by running it on test environment
- For example, if the tests are to be documented for using again in future for regression testing, the test documents will record step by step description of executing the test.

Disadvantages of early test implementation

- Implementing the tests early may have some disadvantages too.
- For example, if Agile lifecycle has been adopted for product development, the code itself may undergo drastic changes between consecutive iterations. This will render the whole test implementation useless.
- In fact, any iterative development lifecycle will affect the code between iterations, even if it is not as drastic as that in the Agile lifecycle.
- This will make pre-defined tests obsolete or require continuous and resource intensive maintenance.
- Similarly, even in case of sequential lifecycles, if the project is badly managed and requirements keep changing even when project is in an advanced state, early test implementation can be rendered obsolete.

Therefore, before starting the test implementation process, Test Manager should consider these important points:

- Software development life cycle being used
- Features that need to be tested
- Probability of change in requirement late into project lifecycle
- Possibility of changes in code between two iterations

Test Execution

The term Test Execution tells that the testing for the product or application needs to be executed in order to obtain the expected result.

After the development phase, the testing phase will take place where the various levels of testing techniques will be carried out and the creation and execution of test cases will be taken place.

Test Execution is the process of executing the tests written by the tester to check whether the developed code or functions or modules are providing the expected result as per the client requirement or business requirement.

Importance of Test Execution:

- **The project runs efficiently:** Test execution ensures that the project runs smoothly and efficiently.
- **Application competency:** It also helps to make sure the application's competency in the global market.
- **Requirements are correctly collected:** Test executions make sure that the requirements are collected correctly and incorporated correctly in design and architecture.
- **Application built in accordance with requirements:** It also checks whether the software application is built in accordance with the requirements or not.

Activities for Test Execution

The following are the 5 main activities that should be carried out during the test execution.

1. **Defect Finding and Reporting:** Defect finding is the process of identifying the bugs or errors raised while executing the test cases on the developed code or modules. If any error appears or any of the test cases failed then it will be recorded and the same will be reported to the respective development team.
2. **Defect Mapping:** After the error has been detected and reported to the development team, the development team will work on those errors and fix them as per the requirement. Once the development team has done its job, the tester team will again map the test cases or test scripts to that developed module or code to run the entire tests to ensure the correct output.

3. Re-Testing: Re-Testing is the process of testing the modules or entire product again to ensure the smooth release of the module or product. In some cases, the new module or functionality will be developed after the product release. In this case, all the modules will be re-tested for a smooth release. So that it cannot cause any other defects after the release of the product or application.

4.Regression Testing: Regression Testing is software testing that ensures that the newly made changes to the code or newly developed modules or functions should not affect the normal processing of the application or product.

5.System Integration Testing: System Integration Testing is a type of testing technique that will be used to check the entire component or modules of the system in a single run. It ensures that the whole system will be checked in a single test environment instead of checking each module or function separately.

Test Execution Process

The test Execution technique consists of three different phases. The three main phases of test execution are the creation of test cases, test case execution, and validation of test results.

1. Creation of Test Cases: The first phase is to create suitable test cases for each module or function. Here, the tester with good domain knowledge must be required to create suitable test cases. The creation of test cases should not be delayed else it will cause excess time to release the product. The created test cases should not be repeated again. It should cover all the possible scenarios raised in the application.

2. Test Cases Execution: After test cases have been created, execution of test cases will take place. The Quality Analyst team will either do automated or manual testing depending upon the test case scenario. The selection of testing tools is also important to execute the test cases.

3. Validating Test Results: After executing the test cases, note down the results of each test case in a separate file or report. Check whether the executed test cases achieved the expected result and record the time required to complete each test case i.e., measure the performance of each test case. If any of the test cases is failed or not satisfied the condition then report it to the development team for validating the code.

Ways to Perform Test Execution

1. **Run test cases:** It is a simple and easiest approach to run test cases on the local machine and it can be coupled with other artifacts like test plans, test suites, test environments, etc.
2. **Run test suites:** A test suite is a collection of manual and automated test cases and the test cases can be executed sequentially or in parallel. Sequential execution is useful in cases where the result of the last test case depends on the success of the current test case.
3. **Run test case execution and test suite execution records:** Recording test case execution and test suite execution is a key activity in the test process and helps to reduce errors, making the testing process more efficient.
4. **Generate test results without execution:** Generating test results from non-executed test cases can be helpful in achieving comprehensive test coverage.
5. **Modify execution variables:** Execution variables can be modified in the test scripts for particular test runs.
6. **Run automated and manual tests:** Test execution can be done manually or can be automated.
7. **Schedule test artifacts:** Test artifacts include video, screenshots, data reports, etc. These are very helpful as they document the results of the past test execution and provide information about what needs to be done in future test execution.
8. **Defect tracking:** Without defect tracking test execution is not possible, as during testing one should be able to track the defects and identify what when wrong and where.

Test Execution Priorities are nothing but prioritizing the test cases depending upon several factors.

- **Complexity:** The complexity of the test cases can be determined by including several factors such as boundary values of test cases, features or components of test cases, data entry of test cases, and how much the test cases cover the given business problem.
- **Risk Covered:** How much risk that a certain test case may undergo to achieve the result. Risk in the form of time required to complete the test case process, space complexity whether it is executed in the given memory space, etc.,
- **Platforms Covered:** It simply tells that in which platform or operating system the test cases have been executed i.e., test cases executed in the Windows OS, Mac OS, Mobile OS, etc.,
- **Depth:** It covers how depth the given test cases cover each functionality or module in the application i.e., how much a given test procedure covers all the possible conditions in a single functionality or module.
- **Breadth:** It covers how the breadth of the given test cases covers the entire functionality or modules in the application i.e., how much a given test procedure covers all the possible conditions in the entire functionality or modules in the product or application.

Test Execution States

The tester or the Quality Analyst team reports or notices the result of each test case and records it in their documentation or file. There are various results raised when executing the test cases. They are

- **Pass:** It tells that the test cases executed for the module or function are successful.
- **Fail:** It tells that the test cases executed for the module or function are not successful and resulted in different outputs.
- **Not Run:** It tells that the test cases are yet to be executed.
- **Partially Executed:** It tells that only a certain number of test cases are passed and others aren’t met the given requirement.
- **Inconclusive:** It tells that the test cases are executed but it requires further analysis before the final submission.
- **In Progress:** It tells that the test cases are currently executed.
- **Unexpected Result:** It tells that all the test cases are executed successfully but provide different unexpected results.

Test Execution Report

The Test Execution Report is nothing but a document that contains all the information about the test execution process. The documentation or the report contains various information. They are:

- Who all are going to execute the test cases?
- Who is doing the unit testing, integration testing, system testing, etc.,
- Who is going to write test cases?
- The number of test cases executed successfully.
- The number of test cases failed during the testing.
- The number of test cases executed today.
- The number of test cases yet to be executed.
- What are the automation test tools used for today's test execution?
- What are the modules/functions testing today?
- Recording the issues while executing the test cases.
- What is today's testing plan?
- What is tomorrow's testing plan?

- Recording the pending plans.
- Overall success rate.
- Overall failure rate.
- Test Summary Report Identifier.
- Summary.
- Variances.
- Comprehensive Assessment.
- Summary of Results.
- Evaluation.
- Summary of Activities.
- Approval.

Guidelines for Test Execution

- Write the suitable test cases for each module of the function.
- Assign suitable test cases to respective modules or functions.
- Execute both manual testing as well as automated testing for successful results.
- Choose a suitable automated tool for testing the application.
- Choose the correct test environment setup.
- Note down the execution status of each test case and note down the time taken by the system to complete the test cases.
- Report all the success status and the failure status to the development team
- Track the test status again for the already failed test cases and report it to the team.
- Highly Skilled Testers are required to perform the testing with less or zero failures/defects.
- Continuous testing is required until success test report is achieved.

Reviews, Inspection and Auditing

- https://abodeqa.com/reviewswalkthrough-and-inspection-in-software-testing/

Advantages

- The hybrid method provides features of both Bottom Up and Top Down methods.
- It is most time reducing method.
- It provides complete testing of all modules.

Disadvantages

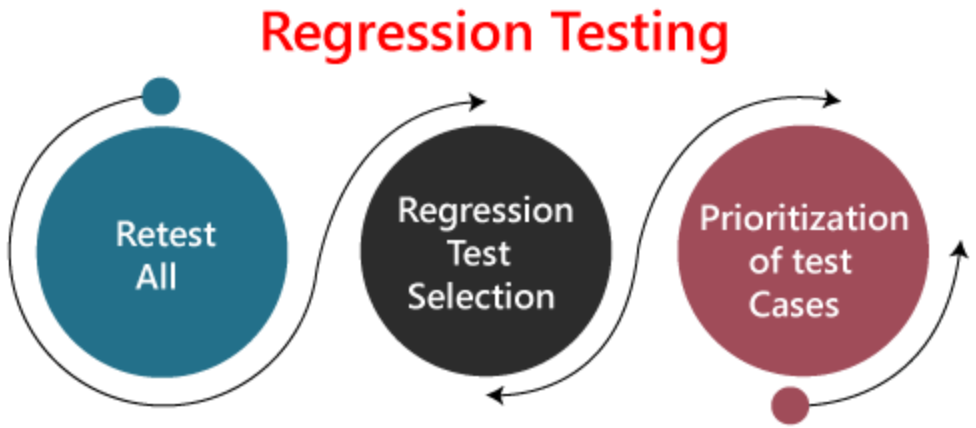
- This method needs a higher level of concentration as the process carried out in both directions simultaneously.
- Complicated method.

Regression Testing

- Regression testing is a black box testing techniques. It is used to authenticate a code change in the software does not impact the existing functionality of the product. Regression testing is making sure that the product works fine with new functionality, bug fixes, or any change in the existing feature.
- Regression testing is a type of software testing. Test cases are re-executed to check the previous functionality of the application is working fine, and the new changes have not produced any bugs.
- Regression testing can be performed on a new build when there is a significant change in the original functionality. It ensures that the code still works even when the changes are occurring. Regression means Re-test those parts of the application, which are unchanged.
- Regression tests are also known as the Verification Method. Test cases are often automated. Test cases are required to execute many times and running the same test case again and again manually, is time-consuming and tedious too.

- When can we perform Regression Testing?
- We do regression testing whenever the production code is modified.
- We can perform regression testing in the following scenario, these are:
- **1. When new functionality added to the application.**
- **Example:**
- A website has a login functionality which allows users to log in only with Email. Now providing a new feature to do login using Facebook.
- **2. When there is a Change Requirement.**
- **Example:**
- Remember password removed from the login page which is applicable previously.
- **3. When the defect fixed**
- **Example:**
- Assume login button is not working in a login page and a tester reports a bug stating that the login button is broken. Once the bug fixed by developers, tester tests it to make sure Login Button is working as per the expected result. Simultaneously, tester tests other functionality which is related to the login button.
-

- **4. When there is a performance issue fix**
- **Example:**
- Loading of a home page takes 5 seconds, reducing the load time to 2 seconds.
- **5. When there is an environment change**
- **Example:**
- When we update the database from MySql to Oracle
- How to perform Regression Testing?
- The need for regression testing comes when software maintenance includes enhancements, error corrections, optimization, and deletion of existing features. These modifications may affect system functionality. Regression Testing becomes necessary in this case.
- Regression testing can be performed using the following techniques:



1. Re-test All:

Re-Test is one of the approaches to do regression testing. In this approach, all the test case suits should be re-executed. Here we can define re-test as when a test fails, and we determine the cause of the failure is a software fault. The fault is reported, we can expect a new version of the software in which defect fixed. In this case, we will need to execute the test again to confirm that the fault fixed. This is known as re-testing. Some will refer to this as confirmation testing.

The re-test is very expensive, as it requires enormous time and resources.

2. Regression test Selection:

•In this technique, a selected test-case suit will execute rather than an entire test-case suit.

•The selected test case suits divided in two cases

- Reusable Test cases.
- Obsolete Test cases.

•Reusable test cases can use in succeeding regression cycle.

•Obsolete test cases can't use in succeeding regression cycle.

- **3. Prioritization of test cases:**
- Prioritize the test case depending on business impact, critical and frequently functionality used. Selection of test cases will reduce the regression test suite.

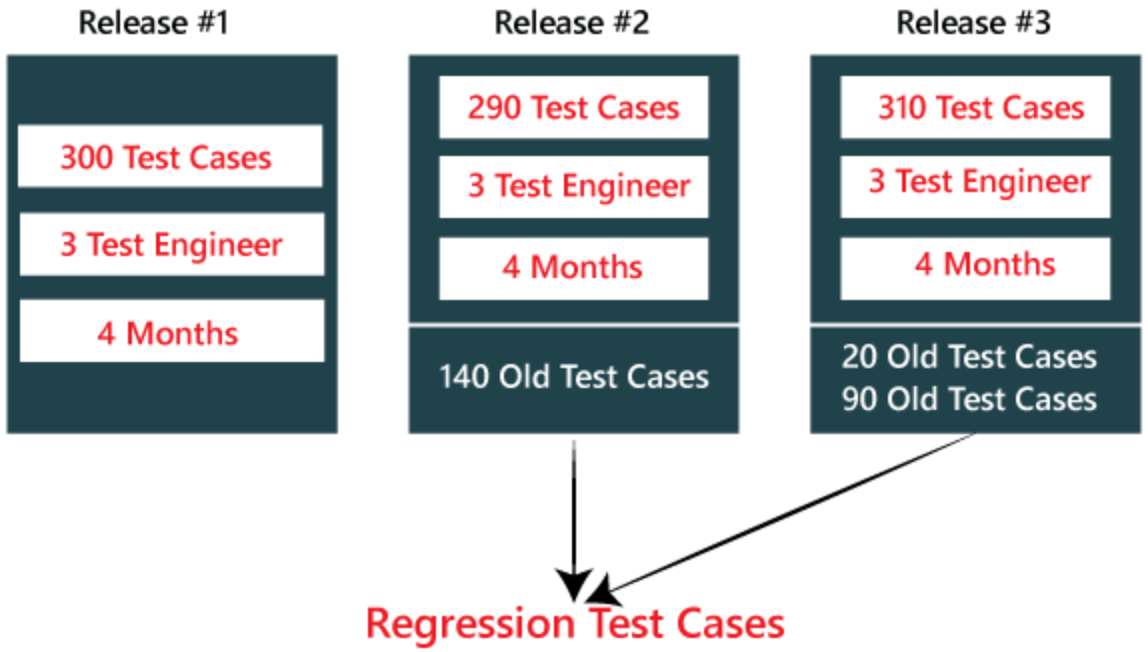
Regression Testing Process

- The regression testing process can be performed across the **builds** and the **releases**.
- Regression testing across the builds
- Whenever the bug fixed, we retest the Bug, and if there is any dependent module, we go for a Regression Testing.

- Regression Testing Across the Release
- The regression testing process starts whenever there is a new Release for same project because the new feature may affect the old elements in the previous releases.
- To understand the regression testing process, we will follow the below steps:
- **Step1**
- There is no regression testing in **Release#1** because there is no modification happen in the Release#1 as the release is new itself.
- **Step2**
- The concept of Regression testing starts from **Release#2** when the customer gives some **new requirements**.
- **Step3**
- After getting the new requirements (modifying features) first, they (the developers and test engineers) will understand the needs before going to the **impact analysis**.
- **Step4**
- After understanding the new requirements, we will perform one round of **impact analysis** to avoid the major risk, but here the question arises who will do the Impact analysis?
- **Step5**
- The impact analysis is done by the **customer** based on their **business knowledge**, the **developer** based on their **coding knowledge**, and most importantly, it is done by the **test engineer** because they have the **product knowledge**.

- **Step6**
- Once we are done with the **impact area**, then the developer will prepare the **impact area (document)**, and the **customer** will also prepare the **impact area document** so that we can achieve the **maximum coverage of impact analysis**.
- **Step7**
- After completing the impact analysis, the developer, the customer, and the test engineer will send the **Reports#** of the impact area documents to the **Test Lead**. And in the meantime, the test engineer and the developer are busy working on the new test case.
- **Step8**
- Once the Test lead gets the Reports#, he/she will **consolidate** the reports and stored in the **test case requirement repository** for the release#1.

- **Step9**
- After that, the Test Lead will take the help of RTM and pick the necessary **regression test case** from the **test case repository**, and those files will be placed in the **Regression Test Suite**.
- **Note:**
- The test lead will store the regression test case in the regression test suite for no further confusion.
- **Regression test suite**: Here, we will save all the impact area test documents.
- **Regression Test Cases**: These are the test cases of the old releases text document which need to be re-executed as we can see in the below image:

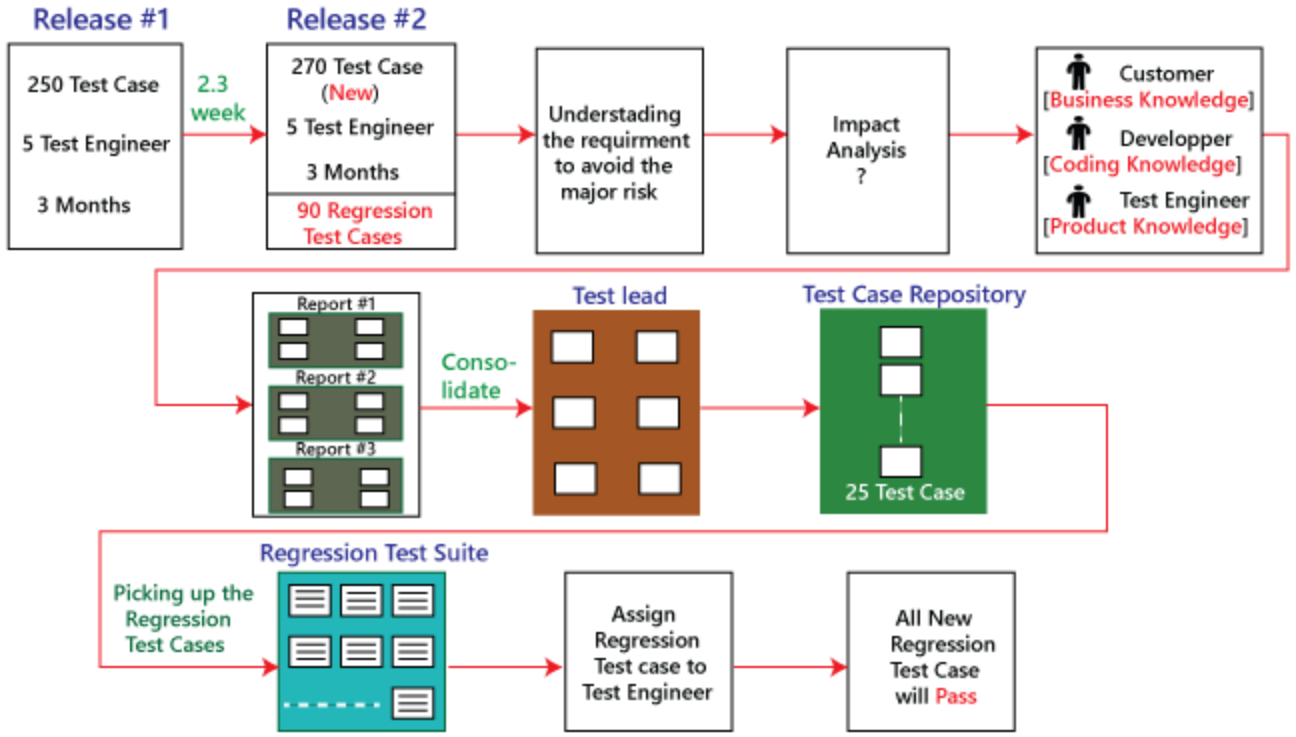


Step10

After that, when the test engineer has done working on the new test cases, the test lead will **assign the regression test case** to the test engineer.

Step11

When all the regression test cases and the new features are **stable and pass**, then check the **impact area using the test case** until it is durable for old features plus the new features, and then it will be handed over to the customer.



Types of Regression Testing

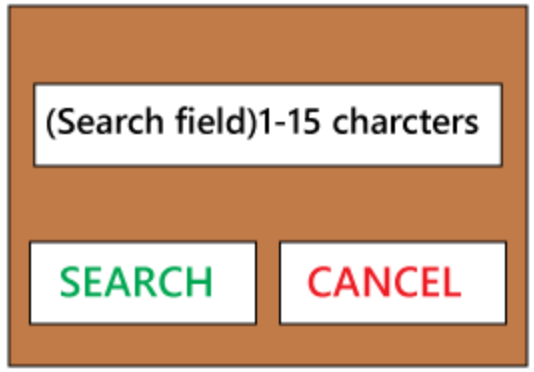
The different types of Regression Testing are as follows:

1.Unit Regression Testing [URT]

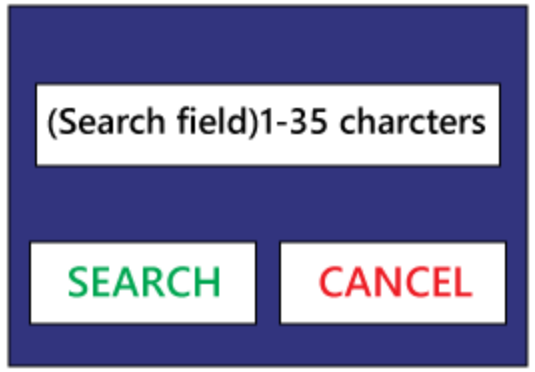
2.Regional Regression Testing[RRT]

3.Full or Complete Regression Testing [FRT]

- 1) Unit Regression Testing [URT]
- In this, we are going to test only the changed unit, not the impact area, because it may affect the components of the same module.
- **Example1**
- In the below application, and in the first build, the developer develops the **Search** button that accepts **1-15 characters**. Then the test engineer tests the Search button with the help of the **test case design technique**.
-



Build 1-B001



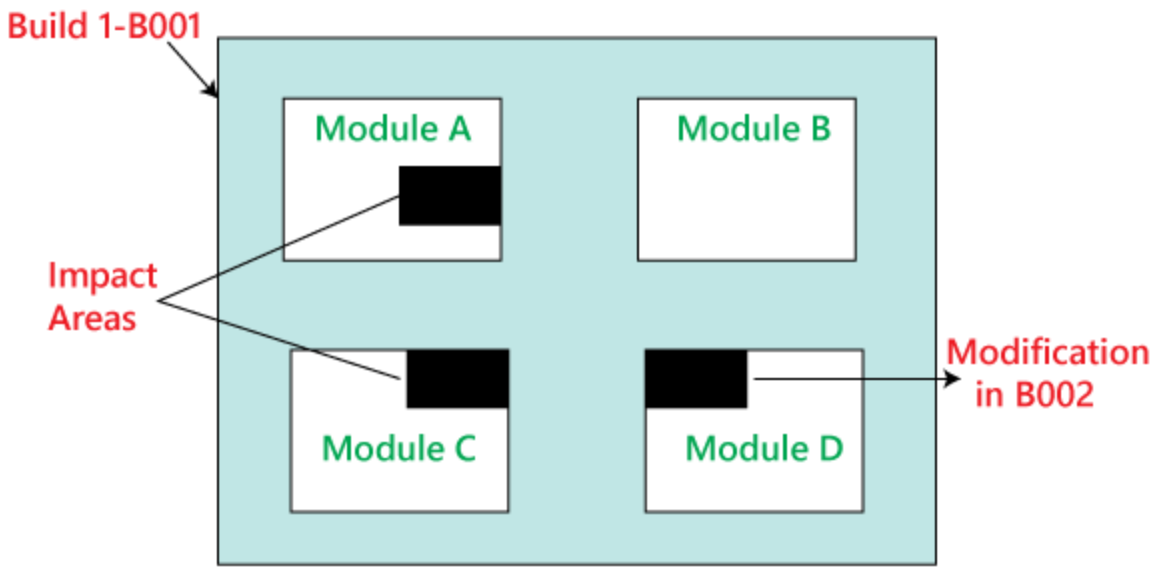
Build 1-B002

- Now, the client does some modification in the requirement and also requests that the **Search button** can accept the **1-35 characters**. The test engineer will test only the Search button to verify that it takes 1-35 characters and does not check any further feature of the first build.
- **Example2**
- Here, we have **Build B001**, and a defect is identified, and the report is delivered to the developer. The developer will fix the bug and sends along with some new features which are developed in the second **Build B002**. After that, the test engineer will test only after the defect is fixed.
- The test engineer will identify that clicking on the **Submit** button goes to the blank page.
- And it is a defect, and it is sent to the developer for fixing it.
- When the new build comes along with the bug fixes, the test engineer will test only the Submit button.
- And here, we are not going to check other features of the first build and move to test the new features and sent in the second build.
- We are sure that fixing the **Submit**button is not going to affect the other features, so we test only the fixed bug.



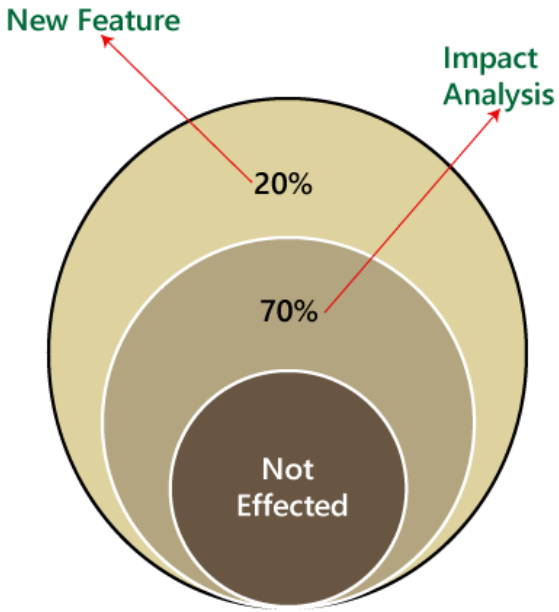
Therefore, we can say that by testing only the changed feature is called the **Unit Regression Testing**.

- 2) Regional Regression testing [RRT]
- In this, we are going to test the modification along with the impact area or regions, are called the **Regional Regression testing**. Here, we are testing the impact area because if there are dependable modules, it will affect the other modules also.
- **For example:**
- In the below image as we can see that we have four different modules, such as **Module A, Module B, Module C, and Module D**, which are provided by the developers for the testing during the first build. Now, the test engineer will identify the bugs in **Module D**. The bug report is sent to the developers, and the development team fixes those defects and sends the second build.



In the second build, the previous defects are fixed. Now the test engineer understands that the bug fixing in Module D has impacted some features in **Module A and Module C**. Hence, the test engineer first tests the Module D where the bug has been fixed and then checks the impact areas in **Module A and Module C**. Therefore, this testing is known as **Regional regression testing.**

- 3) Full Regression testing [FRT]
- During the second and the third release of the product, the client asks for adding 3-4 new features, and also some defects need to be fixed from the previous release. Then the testing team will do the Impact Analysis and identify that the above modification will lead us to test the entire product.
- Therefore, we can say that testing the **modified features** and **all the remaining (old) features** is called the **Full Regression testing**.



- When we perform Full Regression testing?
- We will perform the FRT when we have the following conditions:
- When the modification is happening in the source file of the product. **For example**, JVM is the root file of the JAVA application, and if any change is going to happen in JVM, then the entire JAVA program will be tested.
- When we have to perform n-number of changes.

`

- Mutation Testing
- Mutation testing is a white box method in software testing where we insert errors purposely into a program (under test) to verify whether the existing test case can detect the error or not. In this testing, the mutant of the program is created by making some modifications to the original program.
- The primary objective of mutation testing is to check whether each mutant created an output, which means that it is different from the output of the original program. We will make slight modifications in the mutant program because if we change it on a massive scale than it will affect the overall plan.
- When we detected the number of errors, it implies that either the program is correct or the test case is inefficient to identify the fault.
- Mutation testing purposes is to evaluate the quality of the case that should be able to fail the mutant code hence this method is also known as Fault-based testing as it used to produce an error in the program and that why we can say that the mutation testing is performed to check the efficiency of the test cases.

Types of mutation testing

- Mutation testing can be classified into three parts, which are as follows:
- Decision mutations
- value mutations
- Statement mutations

Decision mutations

- In this type of mutation testing, we will check the design errors. And here, we will do the modification in arithmetic and logical operator to detect the errors in the program.
- Like if we do the following changes in arithmetic operators:
- plus(+)→ minus(-)
- asterisk(*)→ double asterisk(**)
- plus(+)→incremental operator(i++)
- Like if we do the following changes in logical operators
- Exchange P **>** → P**<**, OR P**>=**
- Now, let see one example for our better understanding:

Original Code	Modified Code

if(p >q)
r= 5;
else
r= 15;
	if(p < q)
	r = 5;
	else
	r = 15;

Value mutations

In this, the values will modify to identify the errors in the program, and generally, we will change the following:

- Small value à higher value
- Higher value àSmall value.

For Example:

Original Code	Modified Code

int add =9000008;
int p = 65432;
int q =12345;
int r = (p+ q);
	int mod = 9008;
	int p = 65432;
	int q =12345;
	int r= (p + q);

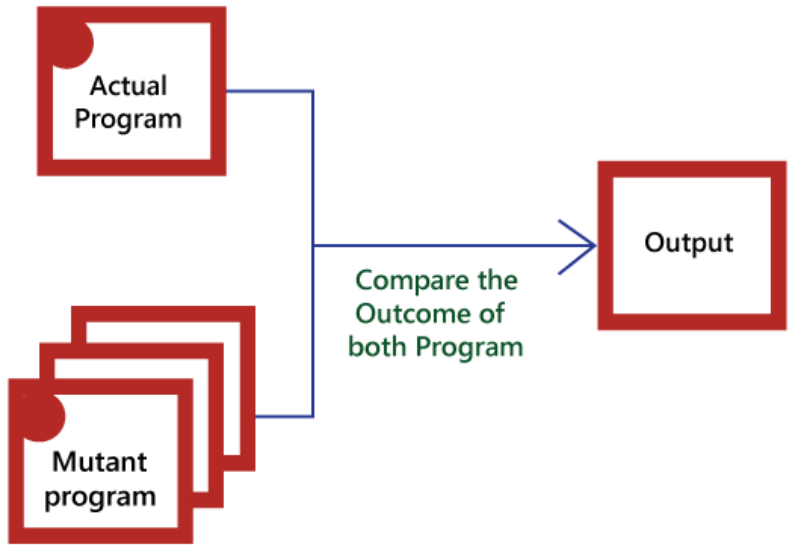
- Statement Mutations
- Statement mutations means that we can do the modifications into the statements by removing or replacing the line as we see in the below example:

Original Code	Modified Code

if(p * q)
r = 15;
else
r = 25;
	if(p* q)
	s = 15;
	else
	s = 25;

How to perform mutation testing

- To perform mutation testing, we will follow the below process:



- In this, firstly, we will add the errors into the source code of the program by producing various versions, which are known mutants. Here every mutant having the one error, which leads the mutant kinds unsuccessful and also validates the efficiency of the test cases.
- After that, we will take the help of the test cases in the mutant program and the actual application will find the errors in the code.
- Once we identify the faults, we will match the output of the actual code and mutant code.
- After comparing the output of both actual and mutant programs, if the results are not matched, then the mutant is executed by the test cases. Therefore the test case has to be sufficient for identifying the modification between the actual program and the mutant program.
- And if the actual program and the mutant program produced the exact result, then the mutant is saved. And those cases are more active test cases because it helps us to execute all the mutants.

- **Web Based Testing**
- Web testing is a software testing technique to test web applications or websites for finding errors and bugs. A web application must be tested properly before it goes to the end-users. Also, testing a web application does not only means finding common bugs or errors but also testing the quality-related risks associated with the application. Software Testing should be done with proper tools and resources and should be done effectively. We should know the architecture and key areas of a web application to effectively plan and execute the testing.
- Testing a web application is very common while testing any other application like testing functionality, configuration, or compatibility, etc. Testing a web application includes the analysis of the web fault compared to the general software faults. Web applications are required to be tested on different browsers and platforms so that we can identify the areas that need special focus while testing a web application.

- **Types of Web Testing:**
- Basically, there are 4 types of web-based testing that are available and all four of them are discussed below:
- **Static Website Testing:** A static website is a type of website in which the content shown or displayed is exactly the same as it is stored in the server. This type of website has great UI but does not have any dynamic feature that a user or visitor can use. In static testing, we generally focus on testing things like UI as it is the most important part of a static website. We check things font size, color, spacing, etc. testing also includes checking the contact us form, verifying URLs or links that are used in the website, etc.
- **Dynamic Website Testing:** A dynamic website is a type of website that consists of both frontend i.e, UI, and the backend of the website like a database, etc. This type of website gets updated or change regularly as per the user's requirements. In this website, there are a lot of functionalities involved like what a button will do if it is pressed, are error messages are shown properly at their defined time, etc. We check if the backend is working properly or not, like does entering the data or information in the GUI or frontend gets updated in the databases or not.
- **E-Commerce Website Testing:** An e-commerce website is very difficult in maintaining as it consists of different pages and functionalities, etc. In this testing, the tester or developer has to check various things like checking if the shopping cart is working as per the requirements or not, are user registration or login functionality is also working properly or not, etc. The most important thing in this testing is that does a user can successfully do payment or not and if the website is secured. And there are a lot of things that a tester needs to test apart from the given things.
- **Mobile-Based Web Testing:** In this testing, the developer or tester basically checks the website compatibility on different devices and generally on mobile devices because many of the users open the website on their mobile devices. So, keeping that thing in mind, we must check that the site is responsive on all devices or platforms.

Points to be Considered While Testing a Website:

- Do all pages are having valid internal and external links or URLs?
- Whether the website is working as per the system compatibility?
- As per the user interface- Does the size of displays are the optimal and the best fit for the website?
- What type of security does the website need (if unsecured)?
- What are the requirements for getting the website analytics, and also controlling graphics, URLs, etc?
- The contact us or customer assistance feature should be added or not on the page, and etc?

Mobile App Testing Tools

- **Best Mobile App Testing Tools for Automation Testing**
- **1) Kobiton**
- Kobiton allows testers an easy-to-use platform to access real devices for manual and automated testing. Kobiton supports complex gestures, ADB shell commands, geo-location, and device connection management. It also offers real-time insight into logs users can explore and download so issues can be identified and resolved.

- **Features:**
- Latest real, cloud-based devices and configurations
- Centralized testing history and data logs for increased collaboration
- Internal Device Lab Management to more effectively utilizes internal devices
- Simplified user experience to streamline test sessions
- On-premises deployment option for extra security
- Support programming language like C#, Java, Ruby, NodeJS, PHP, and Python
- Offers different framework like React Native, Ionic, Electron NativeScript, Xamarin, and Flutter
- Supports Performance Testing, Automation testing, Manual Testing, Functional Testing, and more
- Seamlessly integrates with Travis CI, TeamCity, Jenkins, Azure DevOps, XebiaLabs, Circleci, Jira, and more
- Provides Record-and-replay, Cross-browsing functionality, No-code automation, and real devices testing
- Offers Capabilities, Performance, Customizable Test Cloud, Rich Test Logs, Agile Test Enabler, and Optimized Efficiency
- It provides customer support via Chat, Contact form, and Email
- **Supported Platforms:** iOS, and Android
- **Price:** Plans start at $75 a month.
- **Free Trial:** 14 Days Free Trial (No Credit Card Required)

- **2) testRigor**
- testRigor helps you to directly express tests as executable specifications in plain English. Users of all technical abilities are able to build end-to-end tests of any complexity covering mobile, web, and API steps in one test. Test steps are expressed on the end-user level instead of relying on details of implementation like XPaths or CSS Selectors.

- **Features:**
- Test cases are in English
- Unlimited users & Unlimited tests
- The easiest way to learn automation
- Recorder for web steps
- Email & SMS testing
- Web + Mobile + API steps in one test
- Support programming language like Python, Java, Ruby, JavaScript, PHP, and C#
- Offers different framework like Android, iOS, Angular, React, React Native, and Flutter
- Supports API Testing, Audio Testing, Functional Testing, Security Testing, and more
- Seamlessly integrates with TestRail, Zephyr, XRay, Jira, Azure DevOps, Jenkins, CircleCI, Azure DevOps, and more
- Provides Record-and-replay, Cross-browsing functionality, and No-code automation
- Offers Mocking of API calls, Accessing DBs, 2FA login support, Validation of downloaded files, Generate unique test data, multiple browsers and devices and Reusable Rules
- It provides customer support via Contact Form
- **Supported Platforms:** iOS, and Android
- **Price:** Plans start at $900 a month.
- **Free Trial:** Lifetime Free Public Open Source Plan

- **3) ACCELQ**
- ACCELQ offers AI-powered codeless test automation and management built on a cloud-native platform. ACCELQ provides a unified platform for mobile, web, API, database, and packaged apps. Automation-first, codeless capabilities make it easy to use for testing teams without deep programming expertise. ACCELQ allows businesses to achieve 3x productivity and over 70% savings with its industry-first autonomics-based automation platform.

- **Features:**
- Design, develop and execute mobile test automation with zero setup and no coding.
- Integrated Mobile Cloud Device execution farm with public and private device options for cross-device testing in Plug & Play model.
- Mobile, Web, API, backend and full stack automation in the same unified flow.
- Automation flow recorder, coupled with powerful Natural Language no-code editor.
- Automation that executes across Mobile OS and agnostic of Development frameworks.
- Design-first approach with inbuilt modularity; no need for custom frameworks.
- Robust and sustainable Automation that's significantly low on maintenance.
- Enable Manual testers to Automate without need for programming skills.
- AI-powered Mobile Object handling and self-healing capabilities eliminates Test Flakiness.
- In-sprint automation to align with DevOps and Agile.
- Visual application model for business process validation.
- AI based Automated test case generation and data planning.
- Built-in test management , Version control and governance capabilities.
- Seamless CI/CD integration and natural traceability.
- **Supported Platforms:** Web on Mobile, Native iOS, Native Android , Hybrid Apps.
- **Price:** Plans start at $150 a month.
- **Free Trial:** 14 Days Free Trial (No Credit Card Required)

- **4) Katalon Platform**
- Built on top of Appium and Selenium, Katalon Platform removes the tools' existing steep learning curve and in turn brings a codeless testing experience to users at all scales and expertise. In addition to supporting Android & IOS platforms, testing across OS (Windows, macOS, and Linux) is also available.

- **Features:**
- Simple setup and effortless test creation using record & playback, keywords, images.
- Execute tests locally and remotely on real devices, simulators, or custom cloud-based devices (Sauces Lab, Kobiton, Perfecto, Lambda Test, etc.)
- Flexible test reusability across mobile platforms, API, and Web.
- Reduce maintenance efforts via built-in integration with commonly used project management tools (Jira, Git, Jenkins, etc.)
- Provide insightful test reports of all testing stages for better monitor and collaboration across teams.
- Support programming language like JavaScript, Python, VBScript, JScript, Delphi, C++, and C#
- Offers different framework like Flutter, Xamarin, and React Native
- Supports Visual Testing, Web Testing, API Testing, Mobile Testing, Desktop Testing, and more
- Seamlessly integrates with Azure DevOps, BrowserStack, Circle CI, CodeMagic, Curiosity, GitHub, Google Cloud Build, and more
- Provides Record-and-replay, Cross-browsing functionality, No-code automation, and real devices testing
- Offers Self-Healing, Autonomous Script Generation, Speed, Flexibility & Scalability, Visibility, Innovation & AI and Affordability & ROI
  It provides customer support via Chat, Contact form, and Email
- **Supported Platforms:** iOS, and Android
- **Price:** Plans start at $29 a month. 16% Discount on Yearly Payment.
- **Free Trial:** Lifetime Free Public Open Source Plan

- **5) Perfecto**
- Perfecto is the industry-leading testing cloud for mobile app testing. Prepare your apps for a mobile-first world. Deliver exceptional digital experiences faster and with confidence with Perfecto.

- **Features:**
- Unmatched coverage across platforms and testing scenarios.
- Smart analytics for faster feedback and fixes.
- Unified cloud platform for web and mobile app testing.
- Same-day access to new devices, OSes, and more.
- Enterprise-grade security and scalability.
- Deep technical expertise and support to help you succeed.
- Support programming language like Java, JavaScript, C#, Python, and PHP
- Offers different framework like Flutter, Xamarin, and React Native
- Supports Continuous Testing, Automated Testing, Interactive Testing, Mobile Application Testing, Web Testing, and more
- Seamlessly integrates with Eclipse, IntelliJ, Appium, Espresso, Quantum, TestNG, JUnit, Jira, slack, Perfecto, and more
- Provides Record-and-replay, Cross-browsing functionality, No-code automation, and real devices testing
- Offers Visual Creation, Intelligent Test Creation & Maintenance, Cloud-Based Collaboration, Automatic Device Cleanup, Intelligent Reporting & Debugging, Live Session Sharing, and Secure Local Tunneling
- It provides customer support via Chat, and Contact Form
- **Supported Platforms:** iOS, and Android
- **Price:** Plans start at $99 a month. 16% Discount on Yearly Payment.
- **Free Trial:** 14 Days Free Trial

- **6) TestGrid**
- TestGrid allows users to perform both manual and automated testing of their mobile applications on real devices hosted on-cloud or on your premise in the easiest way.
- You can engage your testing and business teams to build and execute test cases without any pre-requisites of programming knowledge. With TestOS users don’t have to worry about rewriting different test cases but reuse almost all the tests on different versions of the app and on other apps as well. Start with a free plan and upgrade at as low as $29/mo.

- **7) Appium (iOS/Android Testing Tool)**
- Appium is an open source, and a cross platform Mobile Testing Tool for the hybrid and native iOS, it supports Android versions from 2.3 onwards. Appium works like a server running in the background like selenium server. This mobile automation testing tool supports many programming languages, such as Java, Ruby, C# and other which are in the WebDriver library. Appium utilizes WebDriver interface for tests running
- Appium automates Android using the UIAutomator library, which is given by Google as part of the Android SDK. On mobile devices, it can control Safari and Chrome. It can be synchronized with testing framework TestNG. In this case, UI Automator can produce informative and detailed reports, similar to reports generated by Ranorex

- **8) Selendroid**
- Selendroid is a test automation framework that drives off the UI of Android native and hybrid applications (apps) and the mobile web. Using the Selenium 2 client API tests are written.
- **9) Calabash**
- Calabash consists of libraries that allow test-code to programmatically interact with native and hybrid apps.

10) KIF

- KIF mobile app testing tool is objective C based framework and is purely for iOS automated testing. Kif is an mobile automation framework which integrates directly with XCTests. It can be used when business folk are not involved in writing or reading test specs.

- **What Is DevOps?**
- The goal of DevOps is to build better, faster and more responsive software by bringing Development and Operations teams together. DevOps is not a methodology or a suite of tools but a cultural shift to remove the barriers between Dev and Ops in order to meet the need for shorter and more frequent software deliveries. This will allow your organization to respond in a more agile manner to changing business requirements. The DevOps cultural shift depends on continuously optimizing workflow, architecture, and infrastructure in order to deliver high-quality applications.

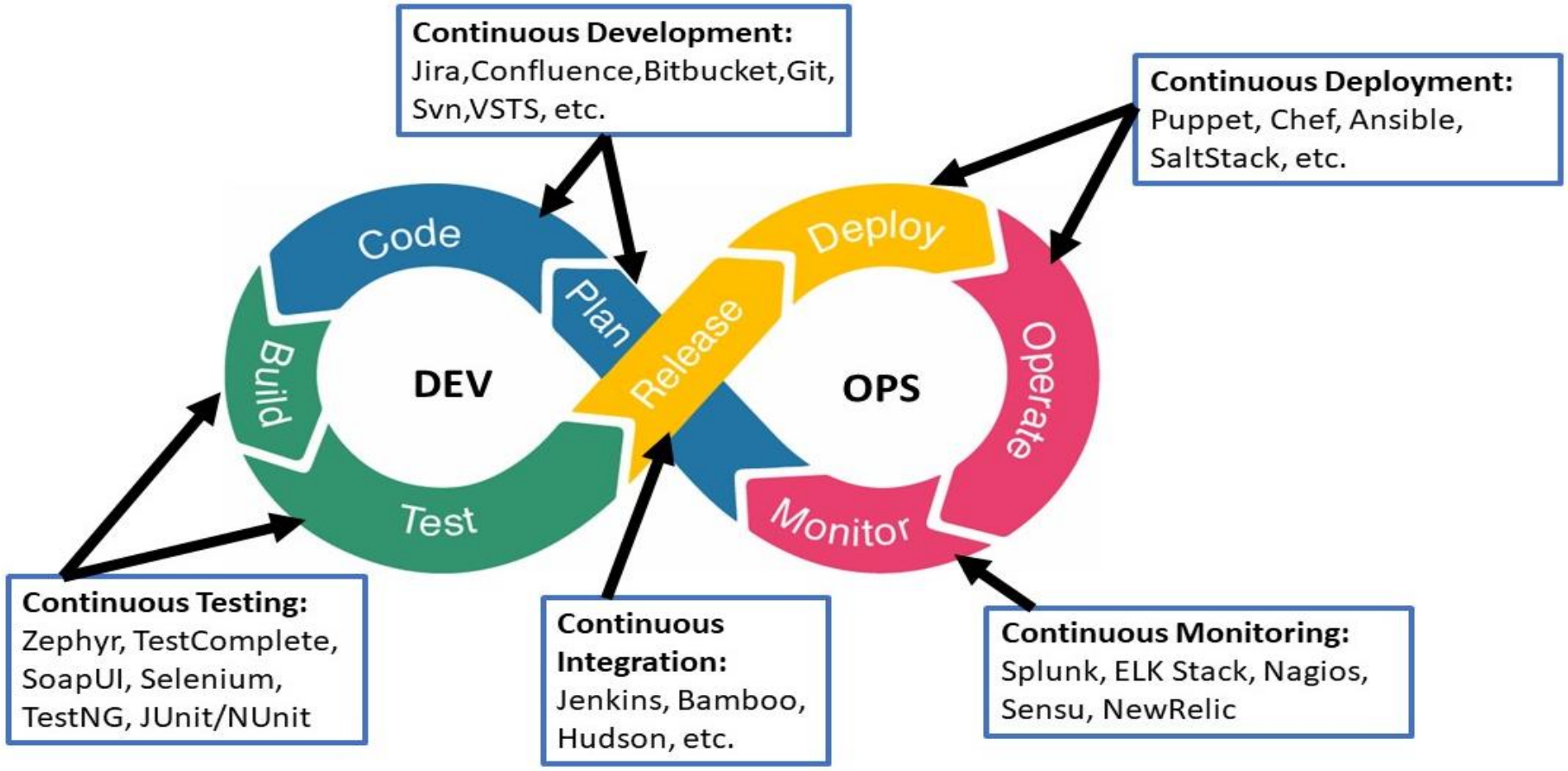
- **From the Agile Model to the DevOps Model**
- Software developed with the Agile Model adheres to the following four basic priorities stated in The Agile Manifesto:

1. Individuals and interactions over processes and tools;
2. Working software over comprehensive documentation;
3. Customer collaboration over contract negotiation; and
4. Responding to change over following a plan.

- Agile development takes a test-first approach, rather than the test-at-the-end approach of traditional development. In agile testing, code is developed and tested in small increments of functionality. Agile project management promotes frequent interaction between an IT department and business users and tries to regularly build and deliver software that meets business users' changing requirements. The flexible approach of the Agile Model helps to ensure a streamlined software development process that responds quickly to customer demand in a wide variety of use cases.

- Agile software development can be implemented with a number of methodologies, including Scrum, Kanban, Scrumban (a mix of Scrum and Kanban), extreme programming, and many more. In the Scrum methodology, the development effort is broken down into a series of short 'sprints,' or iterations, typically lasting 2 to 4 weeks. The goal of each sprint is to create potentially shippable software by adding new or enhancing existing application features.
- The DevOps Model is the 'Agile Model on steroids,' which extends the cross-functional Agile team made up of software designers, testers and developers to include support people from Operations. The focus of this expanded agile team is the overall service or software your company delivers to its customers, instead of just "working software" (the 2nd of the 4 Agile principles listed above).
- Building a successful continuous, two-way DevOps software pipeline between your company and your customers means creating a DevOps culture of collaboration among the various teams involved in software delivery (developers, operations, quality assurance, business analysts, management, and so forth), as well as reducing the cost, time, and risk of delivering software changes by allowing for more incremental updates to applications in production. In practice, this means teams produce software in short cycles, ensuring that the software can be reliably released at any time. Where regular agile teams strive to deliver shippable software in 2 to 4 weeks, the ideal DevOps goal is to deliver code to production daily or every few hours.

- **DevOps Tools**



- software applications pass through five different stages when developed in a DevOps pipeline:

1. Continuous Development
2. Continuous Testing
3. Continuous Integration
4. Continuous Delivery
5. Continuous Monitoring

- Because DevOps is a cultural shift that promotes collaboration between development, QA and operations, there is no one single product that can be considered the definitive DevOps tool. Often a collection of tools, from a variety of vendors, are used in one or more stages of the DevOps toolchain. The DevOps tools used in each stage will vary from organization to organization, based on tools already in place, the skill level of your developers and testers, and the types of applications you're deploying.
- Here is a short list of some the most common DevOps tools and an explanation of how they're used in each stage:

- **Continuous Development with Jira**
- The de facto standard for the agile DevOps software development and testing, Jira Software has powerful collaborative functionality that visually highlights issues as they move through the project workflow, which makes the Jira platform easy to use for planning development, tracking daily work and reporting on project progress.
- **Continuous Testing with Zephyr for Jira**
- Although Jira Software is designed for issue, project and workflow tracking on IT projects, many DevOps teams are also using it for test case management so that development and testing teams can work together in one system. Test issues can be created, executed, tracked and reported on just like any other Jira issue in Zephyr for Jira, which has the same look n' feel of Jira.

- **Continuous Integration with Jenkins**
- Jenkins is a CI/CD server that runs tests automatically every time a developer pushes new code into the source repository. Because CI detects bugs early on in development, bugs are typically smaller, less complex and easier to resolve. Originally created to be a build automation tool for Java applications, Jenkins has since evolved into a multi-faceted platform with over a thousand plug-ins for other software tools. Because of the rich ecosystem of plug-ins, Jenkins can be used to build, deploy and automate almost any software project--regardless of the computer

- **Continuous Delivery/Deployment with Puppet or Chef**
- Automated configuration tools such as Puppet or Chef allow you to avoid the problem of “snowflake servers" in your delivery/deployment environment. “Snowflake servers" are long-running production servers that have been repeatedly reconfigured and modified over time with operating system upgrades, kernel patches, and/or system updates to fix security vulnerabilities. This leads to a server that is unique and distinct, like a snowflake, which requires manual configuration above and beyond that covered by automated deployment scripts.
- Using definition files (called manifests in Puppet, or recipes in Chef) to describe the configuration of the elements of a server is a key best practice in Infrastructure as Code, which allows environments (machines, network devices, operating systems, middleware, etc.) to be configured and specified in a fully automatable format rather than by physical hardware configuration or the use of interactive configuration tools.

- **Continuous Monitoring with Splunk or Elk**
- Both Spunk and Elk are log monitoring tools that allow you to do data analysis on transactions that take place in your IT applications after they've been deployed to ensure uniform performance, availability, security, and user experience. Both Splunk and ELK supply a scalable way to collect and index log files and provide a search interface for users to interact with the data in order to create visualizations such as reports, dashboards or alerts.
- Splunk is a paid service that bills according to the volume of your transactions. The ELK Stack is a set of three open-source products—Elasticsearch, Logstash, and Kibana—all developed and maintained by Elastic.

- **DevOps Test Automation**
- Choosing the right DevOps test automation tool is vitally important for your organization since the right software testing tool in the right hands can foster team collaboration, drive down costs, shorten release cycles, and provide real-time visibility into the status and quality of applications in your DevOps pipeline.
- Test automation works by running a large number of tests repeatedly to make sure an application doesn't break whenever new changes are introduced—at the different Unit-, API- and GUI-levels of the Test Automation Pyramid. In addition to Continuous Integration tools such as Jenkins described above, here are some other useful tools for building, testing and deploying applications automatically when requirements change in order to speed up the release velocity of your pipeline apps.

- **Bamboo**
- Bamboo is a CI/CD server from Atlassian. Like Jenkins and other CI/CD servers, Bamboo allows developers to automatically build, integrate, test and deploy source code. Bamboo is closely connected with other Atlassian tools such as Jira for project management and HipChat for team communication. Unlike Jenkins, which is a free and open source agile automation tool, Bamboo is commercial software that is integrated (and supported) out of the box with other Atlassian products such as Bitbucket, Jira, and Confluence.
- **Selenium**
- Selenium is a suite of different open-source software tools used for automated software testing of web applications across various browsers/platforms. Most often used to create robust, browser-based regression automation suites and tests, Selenium--like Jenkins--has a rich repository of open source tools that are useful for different kinds of automation problems. With support for programming languages like C#, Java, JavaScript, Python, Ruby, .Net, Perl, PHP, etc., Selenium can be used to write automation scripts that run against most modern web browsers.

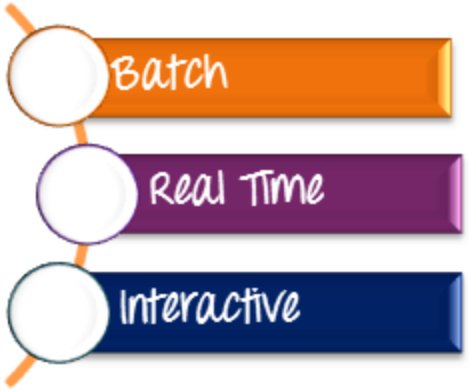
- **TestComplete**
- TestComplete has a powerful and comprehensive set of features for web, mobile, and desktop application testing. In addition to having an easy-to-use record and playback feature, TestComplete allows testers to use JavaScript, VBScript, Python, or C++Script to write test scripts. The tool also an object recognition engine that can accurately detect dynamic user interface elements, which makes it especially useful in applications that have dynamic and frequently changing user interfaces. Since it's a SmartBear product, TestComplete can be integrated easily with other products offered by SmartBear.
- **SoapUI**
- SoapUI is a test automation tool for functional testing, web services testing, security testing, and load testing. Specifically designed for API testing, SoapUI supports both REST and SOAP services. SoapUI provides drag and drops options for creating test suites, test steps and test requests to build complex test scenarios without having to write scripts.

Big Data Testing

- **Big Data Testing** is a testing process of a big data application in order to ensure that all the functionalities of a big data application works as expected. The goal of big data testing is to make sure that the big data system runs smoothly and error-free while maintaining the performance and security.
- Big data is a collection of large datasets that cannot be processed using traditional computing techniques. Testing of these datasets involves various tools, techniques, and frameworks to process. Big data relates to data creation, storage, retrieval and analysis that is remarkable in terms of volume, variety, and velocity. You can learn more about Big Data, Hadoop and MapReduce

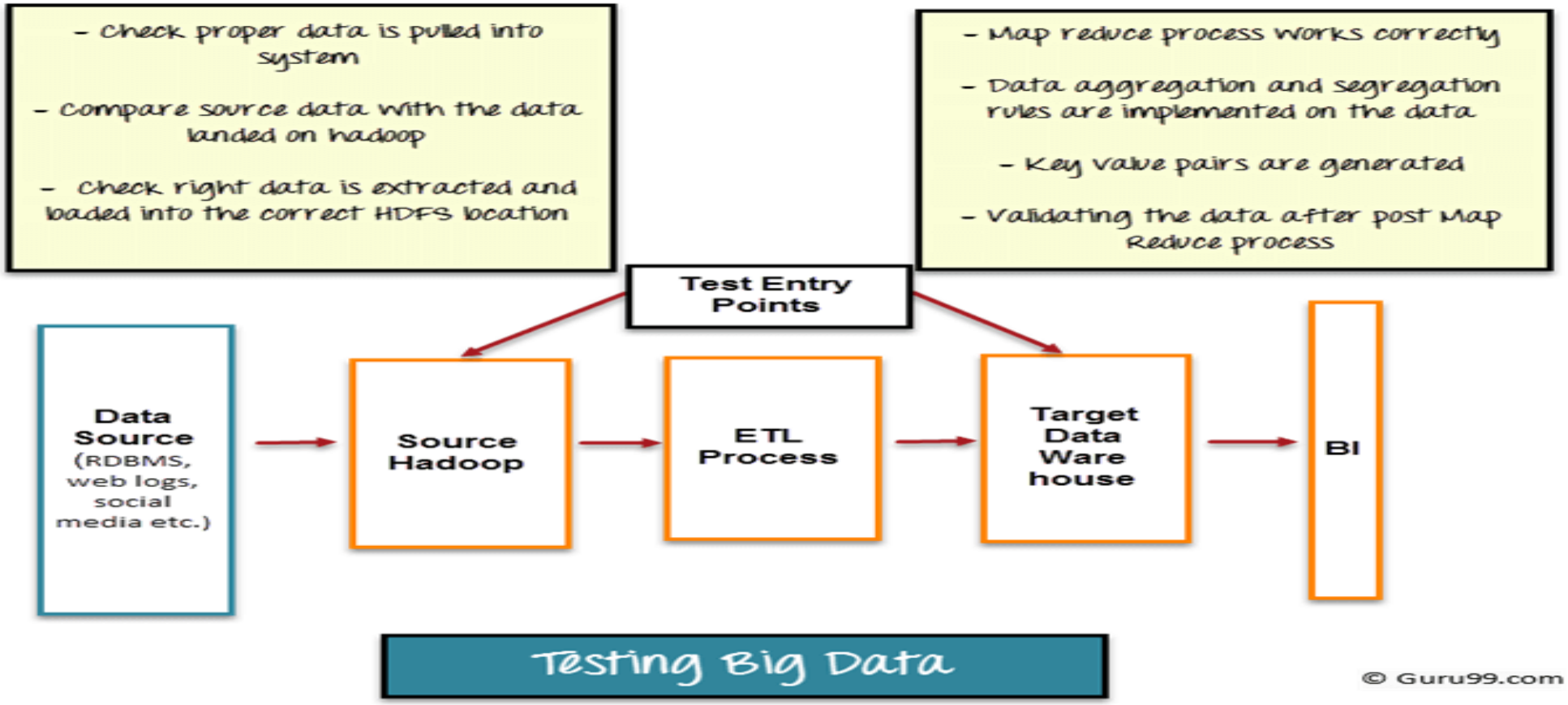
What is Big Data Testing Strategy?

- Testing Big Data application is more verification of its data processing rather than testing the individual features of the software product. When it comes to Big data testing, **performance and functional testing** are the keys.
- In Big Data testing strategy, QA engineers verify the successful processing of terabytes of data using commodity cluster and other supportive components. It demands a high level of testing skills as the processing is very fast. Processing may be of three types



- Along with this, data quality is also an important factor in Hadoop testing. Before testing the application, it is necessary to check the quality of data and should be considered as a part of database testing. **It involves checking various characteristics like conformity, accuracy, duplication, consistency, validity, data completeness,** etc. Next in this Hadoop Testing tutorial, we will learn how to test Hadoop applications.

- **How to test Hadoop Applications**
- The following figure gives a high-level overview of phases in Testing Big Data Applications
-



- Big Data Testing or Hadoop Testing can be broadly divided into three steps
- **Step 1: Data Staging Validation**
- The first step in this big data testing tutorial is referred as pre-Hadoop stage involves process validation.
- Data from various source like RDBMS, weblogs, social media, etc. should be validated to make sure that correct data is pulled into the system
- Comparing source data with the data pushed into the Hadoop system to make sure they match
- Verify the right data is extracted and loaded into the correct HDFS location
- Tools like **Talend**, **Datameer**, can be used for data staging validation
- **Step 2: "MapReduce" Validation**
- The second step is a validation of "MapReduce". In this stage, the Big Data tester verifies the business logic validation on every node and then validating them after running against multiple nodes, ensuring that the
- Map Reduce process works correctly
- Data aggregation or segregation rules are implemented on the data
- Key value pairs are generated
- Validating the data after the Map-Reduce process

- **Step 3: Output Validation Phase**
- The final or third stage of Hadoop testing is the output validation process. The output data files are generated and ready to be moved to an EDW (Enterprise Data Warehouse) or any other system based on the requirement.
- Activities in the third stage include
- To check the transformation rules are correctly applied
- To check the data integrity and successful data load into the target system
- To check that there is no data corruption by comparing the target data with the HDFS file system data

- **Architecture Testing**
- Hadoop processes very large volumes of data and is highly resource intensive. Hence, architectural testing is crucial to ensure the success of your Big Data project. A poorly or improper designed system may lead to performance degradation, and the system could fail to meet the requirement. At least, **Performance and Failover test** services should be done in a Hadoop environment.
- Performance testing includes testing of job completion time, memory utilization, data throughput, and similar system metrics. While the motive of Failover test service is to verify that data processing occurs seamlessly in case of failure of data nodes

- **Performance Testing**
- Performance Testing for Big Data includes two main action
- **Data ingestion and Throughout**: In this stage, the Big Data tester verifies how the fast system can consume data from various data source. Testing involves identifying a different message that the queue can process in a given time frame. It also includes how quickly data can be inserted into the underlying data store for example insertion rate into a Mongo and Cassandra database.
- **Data Processing**: It involves verifying the speed with which the queries or map reduce jobs are executed. It also includes testing the data processing in isolation when the underlying data store is populated within the data sets. For example, running Map Reduce jobs on the underlying HDFS
- **Sub-Component Performance**: These systems are made up of multiple components, and it is essential to test each of these components in isolation. For example, how quickly the message is indexed and consumed, MapReduce jobs, query performance, search, etc.

- **Performance Testing Approach**
- Performance testing for big data application involves testing of huge volumes of structured and unstructured data, and it requires a specific testing approach to test such massive data.



Performance Testing Approach

© Guru99.com

- Performance Testing is executed in this order

1. The process begins with the setting of the Big data cluster which is to be tested for performance
2. Identify and design corresponding workloads
3. Prepare individual clients (Custom Scripts are created)
4. Execute the test and analyzes the result (If objectives are not met then tune the component and re-execute)
5. Optimum Configuration